

Presentation Cheat Sheet: Parallax Simulation Technicals

1. PlanetShader Logic

- **Blending:** Uses a **Three-Layer System** (Surface + Atmosphere + Night Lights). Surface and Atmosphere are merged via **Screen Blend** ($1 - (1-A)*(1-B)$) for glow effects; Atmosphere masks out Night Lights.
- **Shadows:** Standard **Lambertian Lighting** ($N \cdot L$) with a **smoothstep** terminator for soft day-to-night transitions.
- **Stencil Buffer:** Used for **Z-Order Management**. It prevents “Z-fighting” at astronomical scales by prioritizing rendering order (Closest Body = Highest Ref ID) independently of the depth buffer.

2. Coordinate System & Precision

- **Hierarchical Positioning:** Space is split into **1-Million AU “Sectors”**. A position is defined as [Sector Integer] + [Local Double Offset].
- **Floating Point Fix:** By keeping local offsets small relative to the sector origin, we maintain **millimeter precision** even when 100,000 light-years away.
- **Parallax (Motion Depth):**
 - **3-Step Math:**
 1. **Vector:** $\text{StarPos} - \text{PlayerPos}$. Both are relative to the Sun (0,0,0).
 2. **Cull:** Instantly drop objects not in the Camera FOV.
 3. **Normalize:** Lock the star onto the sky sphere (Horizon).
 - **Player Position:** Stored as a **Dual Value** (Sector ID + Local Offset). The system calculates the total offset from the Sun to create the PlayerPos.
 - **Data Types:**
 - * **Global:** Custom Vector3d (**64-bit Double Precision**) to track positions across lightyears without loss of detail.
 - * **Local/Shader:** Standard Vector3 (**32-bit Float**) for final drawing once the object is relative to the player.
 - **Efficiency:** Only ~5-6 CPU/GPU operations per star; allows processing millions per frame.
 - **Performance:** “Quick Culling”
 1. **Performance Limit:** To keep the game smooth, we only pick the best ~100k-200k stars out of 10 million.
 2. **Field of View:** We drop anything the camera isn’t currently looking at (the “front-of-face” check).
- **Origin Shifting:** When the player reaches a sector boundary, the world “shifts” in one frame to center the origin on the player—seamless and jitter-free.

2.1 Vector Math Optimization

- **Dot Product vs. Angle:**
 - **The Concept:** To see if a star is in front of the player, we need to know the **Angle** between where the player looks and where the star is.
 - **The Math:** A **Dot Product** calculation ($A \cdot B$) on unit vectors is mathematically identical to the **Cosine of the Angle** ($\cos$).
 - **Why it’s faster:** Calculating the actual angle (in degrees) requires a slow `acos()` function. Comparing the Dot Product to a pre-calculated “Cosine Threshold” gives the same result using only a few multiplications/additions—crucial for processing 10 million stars.

3. Data Architecture

- **Solar System:** High-precision vectors fetched from **JPL Horizons API**.
- **Asteroids:** ~100k records sourced from **JPL Small-Body Database (SBDB)**.
 - **Calculation:** We calculate the asteroid’s position using **Keplerian Mechanics**:
 1. **The Source:** We fetch **Orbital Elements** (the astronomical “DNA” of the orbit) directly from the **JPL SBDB API**.
 2. **The Path:** These elements define the orbit’s shape (oval) and its tilt in space.
 3. **The Timing:** Using the last known position and current time, we calculate where it is along that oval.

- 4. **Kepler's Laws:** Fast near the Sun, slow further away. This allows for precise position prediction without constant API calls.
- **Stars:** 10 Million stars from the **Gaia Mission DR3** dataset.
- **Asteroids & Stars (Required Columns):**
 - **RA (Right Ascension):** Like “Longitude” in the sky (horizontal position).
 - **Dec (Declination):** Like “Latitude” in the sky (vertical position).
 - **Distance (AU/Parsecs):** Essential for parallax. Closer objects shift more than far ones.
 - **Magnitude (Brightness):** Determines how large/bright the object appears on screen.
- **Binary Storage:** Records are packed into `.bin` files as `float32` (RA, Dec, Dist, Mag). This is much faster to load than text and allows GPU-direct processing.
- **Textures:** Mapped via `planet_materials.json`. Links NAIF IDs to high-res maps: <https://www.solarsystemscope.com/t>

4. Computing & Performance

- **Single-Thread (CPU):** Standard C# loop. Processes objects one-by-one. Easy to debug but limited to 1 core; only used for very small data sets.
- **Burst Jobs (Parallel CPU):**
 - **Architecture:** Work is split across all CPU cores.
 - **Burst Compiler:** Converts C# into ultra-optimized machine code using **SIMD** (Single Instruction, Multiple Data).
 - **Native Data:** Uses `NativeArray` to bypass “Garbage Collection” (GC) spikes, ensuring jitter-free performance.
- **GPU Compute Shaders (Fastest):**
 - **Problem With Stereo Rendering:** `UNITY_SETUP_INSTANCE_ID(v); UNITY_INITIALIZE_VERTEX_OUT`
 - **Massive Parallelism:** Uses thousands of tiny GPU threads to process millions of stars at once.
 - **GPU-Only Loop:** Calculations happen entirely on the graphics card. The results are sent directly to the renderer via **Indirect Drawing**, meaning the CPU never has to touch the star data.

5. HUD & Labels

- **The Canvas:** Acts as the master container for all UI.
 - **Desktop Range:** Runs in **Screen Space (Overlay)**, stuck to your monitor glass.
 - **VR Range:** Runs in **World Space**. It is a physical “plane” floating 0.6m in front of the camera, allowing it to work with 3D depth.
- **VR HUD Tracking:**
 - **Lock-to-Head:** The HUD is parented to the VR Camera to stay in view.
 - **Inverse Scaling:** As the camera’s world scale changes dynamically (when near planets), the HUD uses an `1.0 / CameraScale` calculation to stay a fixed physical size and distance from your eyes.
- **Displayed Values:**
 - **Velocity:** Shown in **km/s** or **c** (speed of light multiples).
 - **Solar Dist:** Distance from the Sun in **km** or **ly** (lightyears).
 - **Proximity:** Real-time distance and name of the closest planet/moon.
- **Label Culling:**
 - **Occlusion:** Labels are hidden if a planet blocks the line of sight (Raycast check).
 - **Overlap Management:** Labels are sorted by distance; if two labels overlap, one is offset vertically or hidden to keep the view clean.
 - **FOV Culling:** Labels outside the camera’s view are deactivated for performance.

6. Input Methods

- **Mouse Look (SimpleMouseLook.cs):**
 - **Desktop:** Standard Pitch/Yaw control with mouse delta.
 - **VR “Artificial Look”:** Uses an intermediate **Rig Pivot**. Since your physical head handles rotation, the controller thumbstick rotates the *entire rig*, allowing you to turn 360° comfortably while seated.
- **Player Movement:**
 - **Move-in-AU:** We never move the Unity Camera in world space. We only update the player’s **Hierarchical Position** (Sectors/AU). This prevents floating-point jitter.

- **Controls:** WASD (Move), QE (Vertical), Mouse/Head (Look).
- **VR Inputs (XRI Toolkit):**
 - **Autopilot:** X Button / Menu Button.
 - **Planet Info:** I Button / Primary Button.
 - **Orbit Mode:** O Button.
 - **Interactions:** Uses Unity’s **XR Interaction Toolkit** for raycasting and menu selection via controller triggers.

7. Custom Scripts & Components

- **Main Scripts:**
 - **SolarSystemParallaxManager.cs:** The core engine. Handles coordinate precision, planet positions, origin shifting, and asteroid mechanics.
 - **StellarParallaxManager.cs:** The star engine. Manages millions of Gaia stars using GPU/Burst and calculates their parallax motion.
 - **SolarSystemUIManager.cs:** The UI Controller. Manages the VR HUD, planet labels, and menu systems.
 - **SimpleMouseLook.cs:** The interaction script. Manages look-controls for both Desktop (Mouse) and VR (Controller Pivot).
- **Key Exposed Properties:**
 - **SolarManager:** Horizon Radius, Proxy Min/Max Size, Max Travel Distance, Orbit Speed.
 - **StellarManager:** Enable Parallax, Star Size/Brightness, Max Stars Per Frame, Use Compute Shader.
 - **UIManager:** Enable VR Mode, Label Color/Size, HUD Position/Scale, Menu Anim Speed.
 - **MouseLook:** Sensitivity, Invert Y, VR Rotation Pivot.

8. Libraries & Assets

- **Key Libraries (Unity Packages):**
 - **Unity Input System:** Handles all input devices (Keyboard, Mouse, VR) through a unified Action-based system.
 - **TextMeshPro (TMP):** Industry standard for sharp, responsive UI text.
 - **Burst & Jobs:** Enables C# to run as fast as C++ for star/asteroid calculations.
 - **XR Interaction Toolkit (XRI):** Provides the VR framework for hand tracking and spatial interaction.
- **Asset Sources:**
 - **Textures:** High-resolution planetary maps from **SolarSystemScope**.
 - **Star Data:** 10 million records from the **ESA Gaia Mission (DR3)**.
 - **Orbital Data:** Ephemeris and physical data from **NASA JPL Horizons**.
 - **UI Assets:** Custom VR hand models and shaders developed specifically for this project.

9. Python Data Pipeline

- **Data Collection (APIs):**
 - **gaia3_dataset_loader.py:** Automated downloader for the **ESA Gaia DR3** star catalog. Filters 1.8 billion stars down to the best 10 million for the simulation.
 - **solar_snapshot_gaia_plus.py:** Communicates with the **NASA JPL Horizons API** to fetch high-precision coordinates for all planets and moons.
 - **asteroid_snapshot_orbital.py:** Fetches orbital “DNA” (elements) for over 100,000 asteroids from the **JPL Small-Body Database**.
- **Data Manipulation (Formatting):**
 - **gaia_csv_to_binary.py:** Converts massive CSV files into high-performance **Binary (.bin)** files for instant loading in Unity.
 - **calculate_shadow_vectors.py:** Pre-calculates the sun-to-planet vectors used by the **PlanetShadow.shader** to orient shadows correctly.
 - **calculate_sgr_a_star.py:** Computes the precise RA/Dec/Distance for **Sagittarius A*** (the Milky Way’s central black hole) to ensure its position is mathematically accurate relative to the solar system.
 - **naif_id_mapper.py:** Cross-references internal NASA ID numbers (NAIF IDs) with human-readable names and material properties.

- `centauri_system_generator.py`: Procedurally generates the Alpha/Proxima Centauri systems based on real stellar parameters.
- **Verification & Testing:**
 - `analyze_coverage.py`: Validates the **Sky Coverage** of the star dataset. It calculates what percentage of the Milky Way is represented and checks for “holes” in the sky coordinates (RA/Dec).